R Workshop – Day 2 – Importing, Cleaning and Preparing Data

Applied R for Statistical Teaching, Research and Reproducible Analysis

Contents

1 R Workshop – Day 2 – Importing, Cleaning and Preparing Data

Session 2 of 7 | Duration: 3 hours Programme: Applied R for Statistical Teaching, Research and Reproducible Analysis

1.1 Learning objectives

By the end of this session you will be able to:

1. Import a CSV file with `readr::read_csv()` and explain why you keep a raw, untouched copy.
2. Inspect structure, names, ranges and missingness before transforming any data.
3. Write logical rules that flag invalid values without silently deleting rows.
4. Standardise inconsistent category labels.
5. Detect and remove duplicate records.
6. Produce a data-quality report, a missingness report and a short data dictionary.
7. Export a clean dataset to `Data/processed/` so later sessions can rely on it.

1.2 Capstone link

Today the **anchor dataset** is introduced. From this point through Day 7, every session works on this same dataset: a synthetic evaluation of a national statistics-lecturer training programme. Today's task is to choose a research question and turn the raw file into a documented, analysis-ready dataset.

Working research question for this workshop (you may adapt the wording): *Does training track (Online, In-Person, Blended) and class attendance predict improvement in lecturers' post-training assessment scores, after accounting for their pre-training score?*

1.3 Segment plan (3 hours)

Segment	Time	Purpose
Opening and recap	10 min	Revisit Day 1 baseline task, introduce the anchor dataset
Concept bridge	25 min	Why raw data stays untouched; what "clean" means
Guided coding	60 min	Import, inspect, flag, standardise, deduplicate
Participant practice	45 min	Apply the pipeline end-to-end on the anchor dataset
Interpretation and reporting	25 min	Write the data-quality narrative
Evidence log and capstone update	15 min	Save outputs, log research question and variables

---

1.4 Part 1 – Concept bridge (Core workshop activity)

1.4.1 The anchor dataset

`Data/raw/anchor_dataset.csv` contains one row per training participant, with deliberately realistic problems: missing values, a handful of out-of-range values, a few duplicate records, and category labels typed in different ways by different data-entry staff (`"Online"`, `"online"`, `"ONLINE "`). This is not a bug in the workshop materials – it is the exercise.

Variable	Type	Description
`participant_id`	character	Unique lecturer identifier
`age`	numeric	Age in years
`gender`	character	Male / Female
`region`	character	Geopolitical zone
`institution_type`	character	Public / Private
`years_teaching`	numeric	Years of teaching experience
`training_track`	character	Online / In-Person / Blended
`attendance`	numeric	Percentage of sessions attended (0-100)
`satisfaction`	numeric	Likert scale, 1 (low) to 5 (high)
`pre_score`	numeric	Pre-training assessment score (0-100)
`post_score`	numeric	Post-training assessment score (0-100)
`completed`	character	Yes / No – finished the programme

1.4.2 Raw versus working data

Never overwrite the raw file, and never edit it by hand. Read it into one object (e.g. `raw_lecturers`) and do all transformation on a second object (e.g. `clean_lecturers`). If a cleaning rule turns out to be wrong, you re-run the script against the untouched raw file – you do not try to remember what you changed.

1.4.3 What "clean" means here

Cleaning is not "make the numbers look nicer." It is a documented, reproducible set of decisions:

- Which values are *impossible* (e.g. attendance of 140%) versus merely *unusual* (e.g. age 24, the youngest in the group)? Only impossible values become `NA`.
- Which category labels are the *same thing typed differently* versus genuinely different categories?
- Which rows are *exact duplicates* of another row, and should one copy be kept?

> Every decision should be visible in the script, not only in your head.

1.5 Part 2 – Guided coding (Core workshop activity)

1.5.1 2.1 Import with `readr`

```
library(tidyverse)

raw_lecturers <- readr::read_csv("Data/raw/anchor_dataset.csv", show_col_types =
↪  FALSE)
```

`read_csv()` (with an underscore, from `readr`) is preferred over base `read.csv()` from today onward: it is faster on larger files, guesses column types more transparently, and returns a tibble that prints sensibly.

1.5.2 2.2 Inspect before you touch anything

```
glimpse(raw_lecturers)
summary(raw_lecturers)
nrow(raw_lecturers)
sum(duplicated(raw_lecturers))
colSums(is.na(raw_lecturers))
```

Look specifically for: implausible minimums/maximums in `summary()`, the duplicate count, and which columns carry the most missingness.

1.5.3 2.3 Flag invalid values (do not delete rows)

```
flagged_lecturers <- raw_lecturers %>%
  mutate(
    invalid_age = !is.na(age) & (age < 18 | age > 100),
    invalid_attendance = !is.na(attendance) & (attendance < 0 | attendance > 100),
    invalid_satisfaction = !is.na(satisfaction) & !satisfaction %in% 1:5,
    invalid_pre_score = !is.na(pre_score) & (pre_score < 0 | pre_score > 100),
    invalid_post_score = !is.na(post_score) & (post_score < 0 | post_score > 100)
  )

flagged_lecturers %>%
  summarise(across(starts_with("invalid_"), sum, na.rm = TRUE))
```

Flagging first means you can count and report invalid values *before* you remove them – this count is exactly what goes into your data-quality report.

1.5.4 2.4 Replace flagged values with `NA`

```
clean_lecturers <- flagged_lecturers %>%
  mutate(
    age = if_else(invalid_age, NA_real_, age),
    attendance = if_else(invalid_attendance, NA_real_, attendance),
    satisfaction = if_else(invalid_satisfaction, NA_real_, satisfaction),
    pre_score = if_else(invalid_pre_score, NA_real_, pre_score),
    post_score = if_else(invalid_post_score, NA_real_, post_score)
  )
```

1.5.5 2.5 Standardise category labels

```
clean_lecturers <- clean_lecturers %>%
  mutate(
    gender = str_trim(gender),
    gender = case_when(
      str_to_lower(gender) %in% c("m", "male") ~ "Male",
      str_to_lower(gender) %in% c("f", "female") ~ "Female",
      TRUE ~ gender
    ),
    region = str_to_title(str_trim(region)),
    institution_type = str_to_title(str_trim(institution_type)),
    training_track = str_trim(training_track),
    training_track = case_when(
      str_to_lower(training_track) == "online" ~ "Online",
      str_to_lower(training_track) == "blended" ~ "Blended",
      str_to_lower(training_track) %in% c("in-person", "in person", "inperson") ~
      ↪  "In-Person",
      TRUE ~ training_track
```

```
    )
  )

count(clean_lecturers, gender)
count(clean_lecturers, training_track)
```

`str_trim()` removes leading/trailing whitespace; `str_to_lower()`/`str_to_title()` normalise case before you compare or display values. `case_when()` reads top to bottom and stops at the first match – always finish with a fallback (`TRUE ~ ...`) so unexpected values are not silently dropped.

1.5.6 2.6 Duplicates

```
n_before <- nrow(clean_lecturers)
clean_lecturers <- clean_lecturers %>% distinct(participant_id, .keep_all = TRUE)
n_duplicates_removed <- n_before - nrow(clean_lecturers)
n_duplicates_removed
```

Deduplicate on the identifier *after* standardising labels – otherwise `"Online"` and `"online"` rows for the same participant might look like different records.

1.5.7 2.7 Data-quality and missingness reports

```
quality_report <- flagged_lecturers %>%
  summarise(
    rows = n(),
    duplicates_removed = n_duplicates_removed,
    invalid_age = sum(invalid_age, na.rm = TRUE),
    invalid_attendance = sum(invalid_attendance, na.rm = TRUE),
    invalid_satisfaction = sum(invalid_satisfaction, na.rm = TRUE),
    invalid_pre_score = sum(invalid_pre_score, na.rm = TRUE),
    invalid_post_score = sum(invalid_post_score, na.rm = TRUE)
  )

missingness_report <- clean_lecturers %>%
  select(-starts_with("invalid_")) %>%
  summarise(across(everything(), ~ sum(is.na(.x)))) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "missing_count")
```

1.5.8 2.8 Export

```
clean_lecturers <- clean_lecturers %>% select(-starts_with("invalid_"))

dir.create("Data/processed", showWarnings = FALSE, recursive = TRUE)
write_csv(clean_lecturers, "Data/processed/anchor_dataset_clean.csv")
write_csv(quality_report, "Data/processed/data_quality_report.csv")
write_csv(missingness_report, "Data/processed/missingness_report.csv")
```

1.6 Part 3 – Participant practice (Core workshop activity)

Open `Starter-Scripts/day2_anchor_cleaning.R`. It already contains the skeleton above with `TODO` markers. Work through each `TODO` yourself rather than copying Part 2 verbatim – in particular:

1. Run the import and inspection steps and write down, in a comment, which columns concerned you most before cleaning.
2. Complete the invalid-value replacement step.
3. Complete the category-standardisation step, then check `count()` on each categorical column until every label set looks correct.
4. Remove duplicates and confirm the row count drop matches what you expected from `sum(duplicated(...))`.
5. Build the two reports and the export.
6. Create `Data/processed/data_dictionary.csv` with one row per variable: `variable`, `type`, `description`, `valid_range_or_levels`.

A facilitator-reviewed solution is in `Solution-Scripts/day2_anchor_cleaning_solution.R`.

1.7 Optional demonstration

- **Reshaping** with `pivot_longer()`/`pivot_wider()` – useful once you start comparing pre- and post-scores side by side as one "time" column rather than two columns.
- **Importing other formats:** `readxl::read_excel()`, `haven::read_sav()` (SPSS), `haven::read_dta()` (Stata), `haven::read_sas()` (SAS). Demonstrate one if participants' own data comes from these formats.

```r
# Optional: reshape pre/post scores into long format for later plotting
clean_lecturers %>%
  select(participant_id, pre_score, post_score) %>%
  pivot_longer(cols = c(pre_score, post_score),
               names_to = "timing", values_to = "score")
```

1.8 Reference or take-home material

- Keeping an audit trail: instead of overwriting values, some teams add a `*_original` column before correcting a value, so the original is always recoverable.
- More validation-rule examples (date ranges, cross-field checks such as `post_score` requiring a non-missing `pre_score`).

1.9 Daily deliverable

A clean anchor dataset, a data-quality report, a missingness report, and a short data dictionary, all saved in `Data/processed/`, plus a one-paragraph statement of your chosen research question.

1.9.1 Evidence log entry

Complete the Day 2 row in `Workshop-Evidence-Log.md`: note your research question, confirm the clean dataset and dictionary are saved, and record one cleaning decision you are unsure about so you can revisit it later.

1.10 Facilitator notes

- The dataset is generated with a fixed random seed, so every participant's raw file is identical – this lets you state exact expected counts (240 unique participants, 6 duplicate rows, around 5% missingness per column) if you want to use them as a checkpoint.
- Watch for participants who delete invalid rows outright instead of flagging then setting to `NA`. Deleting rows silently changes the denominator for every later analysis; this is the single most common and most consequential mistake on Day 2.
- If a participant's `training_track` recode misses a label, `count(clean_lecturers, training_track)` will show it immediately – use this as a live debugging demonstration.